

Act as an expert Senior Python & Machine Learning Instructor. I have attached a Jupyter Notebook (`.ipynb`) implementing Principal Component Analysis (PCA) from scratch.

I do not want high-level summaries or generic documentation. Instead, please provide an exhaustive, line-by-line, mechanical breakdown of every single operational line of code in this file.

For EVERY line of code (especially the mathematical steps, array manipulations, and plotting configurations), please address the following structural points:

1. UNDERLYING MECHANICS & ARGUMENTS: What do the specific function parameters or arguments mean at a low level? (For example, if a line reads `axis=0`, `full\_matrices=False`, or `[::-1]`, explain exactly what that specific configuration tells the computer to do step-by-step).
2. STRUCTURAL & SHAPE TRANSFORMATION: What are the input array shapes and data types before this line executes, and what is the exact output shape/data type immediately after it executes? (Explicitly use matrix dimensions like (N, D), (D, D), etc., to show how data is being stretched, collapsed, or transformed).
3. BEHAVIORAL ALTERNATIVES: What would happen to the code's behavior, execution flow, data shapes, and logic if those specific parameters were changed to other valid/invalid states? (Provide counter-examples of alternative parameter values and explain why they would either alter the outcome or break the script).
4. ALGORITHMIC & MATH JUSTIFICATION: Why is this exact configuration mathematically or structurally mandatory for the algorithm to function properly?

Please maintain this deep, parameter-focused engineering perspective throughout your entire line-by-line response.

Here is an exhaustive, parameter-focused engineering review of your custom Principal Component Analysis (PCA) scratch implementation. We will systematically parse every single operational line within the class definition, execution methods, and diagnostic visualization code according to your low-level structural guidelines.

Part 1: Class Initialization & Hyperparameters

Line: `def __init__(self, n_components=2, method="svd"):`

1. Underlying Mechanics & Arguments:

- `self`: The memory pointer referencing the specific instance of the PCA class being constructed.

- `n_components=2`: An integer parameter specifying the target dimensionality of the lower-dimensional subspace onto which the data will be projected. It defaults to 2 to facilitate immediate 2D visualization.
- `method="svd"`: A string parameter acting as an algorithmic flag. It dictates whether the underlying linear algebra routines will utilize Singular Value Decomposition ("`svd`") or the explicit Eigenvalue Decomposition of the empirical covariance matrix ("`eig`").

2. Structural & Shape Transformation:

- **Before**: Class namespace template execution.
- **After**: Instantiates a state container within the Python heap memory. No data arrays are bound or transformed at this stage. The scalar hyperparameter variables are registered.

3. BehaviorAL Alternatives:

- If `n_components` is set to an invalid type (e.g., a float like `0.5` or a negative value), the initialization will pass without an immediate crash. However, downstream array slicing mechanisms (e.g., `[:self.n_components]`) will raise a `TypeError: slice indices must be integers or None` or yield empty arrays.
- If `method` is passed as an unsupported string (e.g., "`als`" or "`nipals`"), no error occurs here, but it will trigger a pre-programmed `ValueError` block during execution of the `.fit()` method.

4. Algorithmic & Math Justification:

- This line establishes structural blueprints. Restricting or tracking target dimensionality is essential because PCA assumes the data can be compressed into a lower-dimensional orthogonal basis while minimizing information loss. The choice of `method` determines whether the calculation optimizes for numerical stability ($O(N \cdot D^2)$ via SVD) or explicit structural modeling of feature variance via the Covariance Matrix ($O(D^3)$ via EVD).

Lines:

```
self.n_components = n_components
self.method = method
self.components = None
self.mean = None
self.explained_variance_ratio_ = None
```

1. Underlying Mechanics & Arguments:

- Assigns the function argument values to instance properties within the

object's local scope (`self`).

- Explicitly binds the tracking attributes (`self.components`, `self.mean`, `self.explained_variance_ratio_`) to `None`. This initializes the class fields as state placeholders prior to data consumption.

2. Structural & Shape Transformation:

- Alters the internal dictionary (`self.__dict__`) of the instance. No tensor allocations are performed yet. Shapes do not apply to these attributes until the calculation phases run.

3. BehaviorAL Alternatives:

- Omitting the initialization of fields like `self.components = None` would mean that if a user called `.transform()` before calling `.fit()`, Python would throw an `AttributeError: 'PCA' object has no attribute 'components'` instead of allowing a clean custom validation check.

4. Algorithmic & Math Justification:

- Encapsulating state parameters ensures adherence to standard estimator design patterns. These placeholders must exist so that subsequent execution matrices (the projection matrix W , empirical center vector μ , and eigenvalues λ) can be stored and used as static configurations during testing or inference passes on unseen data.

Part 2: The Fitting Phase (`.fit(self, X)`)

Line: `def fit(self, X):`

1. Underlying Mechanics & Arguments:

- Defines the primary optimization routine of the transformer model.
- `X`: The input data matrix. It must be structured as a contiguous object capable of evaluation by the NumPy numerical subsystem.

2. Structural & Shape Transformation:

- **Input:** An array-like object of shape (N, D) , where N denotes the number of observations (samples) and D represents the original feature space dimensionality. The standard data type is `float64` or `float32`.
- **Output:** The method does not return a new object (implicitly returns `None` or modifies the state in place). The state changes internally.

3. BehaviorAL Alternatives:

- Passing a 1D vector of shape $(N,)$ instead of a 2D matrix will break downstream mean subtraction operations due to a breakdown in standard dimension alignment rules.

4. Algorithmic & Math Justification:

- Formulating the input as a strict 2D structure of rows (samples) and columns (features) is mandatory. The core operations of linear algebra assume that rows signify unique observations and columns represent random variables whose joint distributions are under analysis.

Line: `self.mean = np.mean(X, axis=0)`

1. Underlying Mechanics & Arguments:

- `np.mean(...)`: Computes the arithmetic mean along a specified axis.
- `axis=0`: Tells the computer to collapse the row dimension (N) by summing elements down each individual column and dividing by N .

2. Structural & Shape Transformation:

- **Input:** `X` of shape `(N, D)` and type `float64`.
- **Output:** `self.mean` of shape `(D,)` and type `float64`. The row dimension has been completely compressed.

3. BehaviorAL Alternatives:

- If changed to `axis=1`, the operation collapses the feature columns, yielding a vector of shape `(N,)` that calculates the mean value per sample across all features. This would cause an operational failure on the next line during centering, throwing an exception: `ValueError: operands could not be broadcast together with shapes (N,D) (N,)`.
- If `axis` is omitted entirely (`None`), the method flattens the matrix, returning a single scalar value representing the global average of all entries, destroying individual feature indicators.

4. Algorithmic & Math Justification:

- PCA requires that data features have an expected value of zero ($\mathbb{E}[X] = 0$). Centering places the coordinate system origin exactly at the multidimensional center of mass. This forces the downstream covariance calculation to simplify directly to a matrix multiplication product ($X^T X$) without requiring an explicit correction offset.

Line: `X_centered = X - self.mean`

1. Underlying Mechanics & Arguments:

- Performs element-wise subtraction. This operation relies on NumPy's internal broadcasting engine. The computer stretches the vector `self.mean` of shape `(D,)` into an implicit virtual matrix of shape `(N, D)` by duplicating the mean row vector N times down the vertical axis to

match the dimensions of `X`.

2. Structural & Shape Transformation:

- **Inputs:** `X` of shape `(N, D)`; `self.mean` of shape `(D, )`.
- **Output:** `X_centered` of shape `(N, D)`. The structural dimensions are preserved, but the distribution of values shifts along each feature axis.

3. BehaviorAL Alternatives:

- If `self.mean` had an incompatible shape—such as `(N, )` due to an incorrect axis parameter choice—NumPy's broadcasting rules would fail, causing a fatal structural shape mismatch error.

4. Algorithmic & Math Justification:

- Mathematically, this operation updates each data point: $x_{ij} \leftarrow x_{ij} - \mu_j$. This is necessary because if the data mean is not zero, the first principal component will project directly toward the global center of mass rather than tracking the direction of maximum variance.

Part 3: Algorithmic Branching — Method 1: "eig"

Line: `if self.method == "eig":`

1. Underlying Mechanics & Arguments:

- Evaluation of an internal string configuration flag to route execution to the Eigenvalue Decomposition pathway.

2. Structural & Shape Transformation:

- Evaluates to a boolean primitive (`True` or `False`). Does not modify any matrix dimensions.

3. BehaviorAL Alternatives:

- If misspelled or passed in an alternate case without preprocessing (e.g., `"Eig"`), this evaluation fails, sending execution downstream to the else-block.

4. Algorithmic & Math Justification:

- Governs runtime control flow based on whether the user wants to compute eigenvectors via the explicit empirical covariance matrix or through the data matrix layout itself.

Line: `covariance_matrix = np.cov(X_centered, rowvar=False)`

1. Underlying Mechanics & Arguments:

- `np.cov(...)`: Computes the sample covariance matrix.
- `rowvar=False`: This parameter tells the routine that columns represent individual random variables, and rows represent observations.

2. Structural & Shape Transformation:

- **Input:** `X_centered` of shape `(N, D)`.
- **Output:** `covariance_matrix` of shape `(D, D)`. This creates a symmetric, square matrix representing the pairwise relationships between all features.

3. BehaviorAL Alternatives:

- If `rowvar=True` (the default value in NumPy), the computer treats every row as a feature and every column as a sample. The output matrix shape would balloon to `(N, N)`. If $N \gg D$, this can cause the system to run out of memory or compute completely incorrect components based on sample dimensions instead of feature space features.

4. Algorithmic & Math Justification:

- Mathematically, this calculates:

$$\Sigma = \frac{1}{N-1} X_{\text{centered}}^T X_{\text{centered}}$$

This transformation maps variance data into a square linear operator space ($D \times D$). The diagonal entries Σ_{ii} contain the isolated feature variances, while off-diagonal elements Σ_{ij} express linear feature relationships. This matrix forms the foundation for eigenvector tracking.

Line: `eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)`

1. Underlying Mechanics & Arguments:

- `np.linalg.eigh(...)`: A specialized mathematical solver optimized for complex Hermitian or real symmetric matrices. It uses underlying LAPACK routines (like DSYEVD) to find solutions.

2. Structural & Shape Transformation:

- **Input:** `covariance_matrix` of shape `(D, D)`.
- **Output:**
 - `eigenvalues`: A 1D vector of shape `(D,)` containing real scalar roots.
 - `eigenvectors`: A 2D square column-orthogonal matrix of shape `(D, D)`. Each column `eigenvectors[:, i]` corresponds to the scalar entry `eigenvalues[i]`.

3. BehaviorAL Alternatives:

- If `np.linalg.eig` (the standard asymmetric solver) is used instead of `.eigh`, the algorithm runs slower because it doesn't exploit symmetry. It can also introduce small imaginary components due to rounding errors (e.g., `1.2e-16j`), which will crash down-stream plotting functions.

4. Algorithmic & Math Justification:

- This solves the characteristic equation:

$$\Sigma v = \lambda v$$

Because Σ is symmetric, the Spectral Theorem ensures that all eigenvalues λ are real and all eigenvectors $\mathcal{U}$ are orthogonal. These vectors represent the candidate axis directions for our new coordinate system.

Line: `idx = np.argsort(eigenvalues)[::-1]`

1. Underlying Mechanics & Arguments:

- `np.argsort(...)`: Returns an array of integer indices that would sort the `eigenvalues` vector in ascending order.
- `[::-1]`: An extended slicing mechanism. It steps through the array backwards with a stride of `-1`, reversing the sorted indices to create a descending order sequence.

2. Structural & Shape Transformation:

- **Input:** `eigenvalues` of shape `(D,)`.
- **Output:** `idx` of shape `(D,)` containing integer tracking values ranging from 0 to `D-1`.

3. BehaviorAL Alternatives:

- Removing `[::-1]` leaves the indices sorted in ascending order. This means the algorithm would select the columns with the least amount of variance first, producing the exact opposite of standard PCA.

4. Algorithmic & Math Justification:

- Eigenvalues represent the variance magnitude captured along their corresponding eigenvector axes. Because the objective of PCA is to maximize variance retention, components must be prioritized from largest eigenvalue to smallest.

Line: `eigenvalues = eigenvalues[idx]`

1. Underlying Mechanics & Arguments:

- Performs advanced integer array indexing. The computer reorders the items in the `eigenvalues` array to match the sequence defined by the index map `idx`.

2. Structural & Shape Transformation:

- **Input:** Original `eigenvalues` of shape `(D, )`; `idx` of shape `(D, )`.
- **Output:** Sorted `eigenvalues` of shape `(D, )`, arranged in descending numerical order.

3. BehaviorAL Alternatives:

- Omitting this step leaves the eigenvalues unsorted while the eigenvectors get sorted in the next line. This decouples the eigenvalues from their corresponding eigenvectors, corrupting the variance calculations.

4. Algorithmic & Math Justification:

- Keeping these arrays aligned is necessary because the variance ratios computed later depend directly on tracking sorted eigenvalues.

Line: `eigenvectors = eigenvectors[:, idx]`

1. Underlying Mechanics & Arguments:

- Uses 2D slicing combined with integer array indexing. The `:` operator selects all rows, while `idx` reorders the columns of the matrix to match the descending sequence of eigenvalues.

2. Structural & Shape Transformation:

- **Input:** `eigenvectors` of shape `(D, D)`.
- **Output:** Permuted `eigenvectors` matrix of shape `(D, D)`, where the columns are now ordered by descending variance.

3. BehaviorAL Alternatives:

- If indexed incorrectly as `eigenvectors[idx, :]`, the operation shuffles the coordinate components *within* each eigenvector rather than sorting the vectors themselves. This destroys the spatial orientation of the new basis.

4. Algorithmic & Math Justification:

- This step reorganizes our new basis space so that the most important directional axes (the principal components) are shifted to the front of the matrix layout.

Line: `self.components = eigenvectors[:, :self.n_components].T`

1. Underlying Mechanics & Arguments:

- `[:, :self.n_components]`: Retains all feature rows while slicing out the first K columns (where $K = \text{n_components}$).
- `.T`: Transposes the sliced sub-matrix, swapping its rows and columns.

2. Structural & Shape Transformation:

- **Input:** Ordered eigenvectors of shape `(D, D)`.
- **Slicing Output:** Shape `(D, K)`.
- **Transposition Output:** `self.components` of shape `(K, D)`.

3. BehaviorAL Alternatives:

- Forgetting to apply the `.T` transpose operation leaves the array shape at `(D, K)`. This will cause a shape mismatch error during the matrix multiplication in the `.transform()` method: `(N, D) x (D, K)` works, but `(N, D) x (K, D)` will fail.

4. Algorithmic & Math Justification:

- Transposing maps the projection vectors into a standard row-space matrix layout (W). This allows us to project future data matrices using a clean matrix product:

$$Z = X_{\text{centered}} \cdot W^T$$

Line: `total_variance = np.sum(eigenvalues)`

1. Underlying Mechanics & Arguments:

- `np.sum(...)`: Sums all scalar entries in the 1D array to calculate the total variance of the dataset.

2. Structural & Shape Transformation:

- **Input:** `eigenvalues` vector of shape `(D,)`.
- **Output:** `total_variance`, an isolated 64-bit scalar value.

3. BehaviorAL Alternatives:

- If the input data is not centered correctly before this step, the total variance value will be inflated by the distance from the coordinate origin, leading to inaccurate variance metrics.

4. Algorithmic & Math Justification:

- The sum of the eigenvalues of a covariance matrix equals its trace:

$$\text{Tr}(\Sigma) = \sum_{i=1}^D \sigma_i^2$$

This value represents the total continuous variance present across all original dimensions.

Line: `self.explained_variance_ratio_ = eigenvalues[:self.n_components] / total_variance`

1. Underlying Mechanics & Arguments:

- `eigenvalues[:self.n_components]`: Slices out the top K eigenvalues.
- `/ total_variance`: Divides each element by the total scalar variance using an element-wise vector-scalar operation.

2. Structural & Shape Transformation:

- **Inputs:** `eigenvalues` slice of shape $(K,)$; `total_variance` scalar.
- **Output:** `self.explained_variance_ratio_` vector of shape $(K,)$ containing float values between 0.0 and 1.0 .

3. BehaviorAL Alternatives:

- If `total_variance` is zero (which can happen if the dataset consists of identical duplicated rows), this line will trigger a runtime divide-by-zero warning and fill the array with `nan` values.

4. Algorithmic & Math Justification:

- This calculates the percentage of total variance captured by each individual principal component:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^D \lambda_j}$$

This metric provides an explicit measure of how much information is retained within the low-dimensional subspace.

Part 4: Algorithmic Branching — Method 2: "svd"

Line: `elif self.method == "svd":`

1. Underlying Mechanics & Arguments:

- Checks the routing configuration to execute the Singular Value

Decomposition (SVD) pathway.

2. Structural & Shape Transformation:

- Evaluates to a boolean primitive (True or False). Does not modify any matrix dimensions.

3. BehaviorAL Alternatives:

- If the condition is met, execution enters the SVD block, skipping the covariance matrix construction steps entirely.

4. Algorithmic & Math Justification:

- SVD provides a more numerically stable way to compute principal components because it works directly on the centered data matrix without squaring values to build a covariance matrix.

Line: `U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)`

1. Underlying Mechanics & Arguments:

- `np.linalg.svd(...)`: Factorizes the matrix using LAPACK routines (like DGESDD).
- `full_matrices=False`: Directs the solver to compute the economy-size (or thin) decomposition instead of the full square matrix system.

2. Structural & Shape Transformation:

- **Input:** `X_centered` of shape (N, D) .
- **Outputs (assuming $N \geq D$):**
 - `U`: Left singular vectors matrix of shape (N, D) .
 - `S`: 1D array of singular values of shape $(D,)$, sorted in descending order.
 - `Vt`: Right singular vectors matrix of shape (D, D) .

3. BehaviorAL Alternatives:

- If `full_matrices=True`, the shape of `U` expands to a full square matrix of (N, N) . If the dataset contains millions of rows ($N = 1,000,000$), allocating an $N \times N$ matrix will exhaust system memory and crash the program.

4. Algorithmic & Math Justification:

- SVD factorizes the matrix as:

$$X_{\text{centered}} = USV^T$$

The right singular vectors (V^T , returned here as `Vt`) correspond exactly to

the eigenvectors of the covariance matrix ($X^T X$). This allows us to find our principal components directly without the risk of numerical precision loss inherent in computing the covariance matrix explicitly.

Line: `self.components = Vt[:self.n_components]`

1. Underlying Mechanics & Arguments:

- Slices out the first K rows of the right singular matrix `Vt`.

2. Structural & Shape Transformation:

- **Input:** `Vt` matrix of shape `(D, D)`.
- **Output:** `self.components` matrix of shape `(K, D)`.

3. BehaviorAL Alternatives:

- If sliced incorrectly along columns as `Vt[:, :self.n_components]`, the output shape becomes `(D, K)`. This selects truncated versions of all features instead of selecting the top K principal component vectors.

4. Algorithmic & Math Justification:

- Because LAPACK's SVD implementation naturally returns `Vt` pre-sorted by singular value magnitude, the top K rows correspond directly to the orthogonal axes that capture the highest variance.

Line: `eigenvalues = (S ** 2) / (X_centered.shape - 1)`

1. Underlying Mechanics & Arguments:

- `S ** 2`: Squares each singular value element-wise.
- `/ (X_centered.shape - 1)`: Divides the squared values by the degrees of freedom ($N - 1$), where `X_centered.shape` extracts the total number of sample rows (N).

2. Structural & Shape Transformation:

- **Input:** `S` array of shape `(D,)`.
- **Output:** `eigenvalues` transformation array of shape `(D,)`.

3. BehaviorAL Alternatives:

- Dividing simply by N instead of $N - 1$ applies a biased estimator approach. While this won't break the code logic, it will create a discrepancy between the SVD output values and the explicit covariance method ("`eig`").

4. Algorithmic & Math Justification:

- The eigenvalues λ of the covariance matrix are directly related to the

singular values S of the data matrix by the equation:

$$\lambda_i = \frac{s_i^2}{N - 1}$$

This conversion step ensures that both algorithmic pathways arrive at identical variance profiles.

Line: `total_variance = np.sum(eigenvalues)`

1. Underlying Mechanics & Arguments:

- Sums the derived eigenvalue vector entries to compute the total dataset variance.

2. Structural & Shape Transformation:

- **Input:** eigenvalues of shape `(D, )`.
- **Output:** `total_variance` scalar value.

3. BehaviorAL Alternatives:

- If executed on uncentered data, this step returns an incorrect total variance value because the singular values would capture the magnitude of the data's distance from the origin rather than its internal variance.

4. Algorithmic & Math Justification:

- Summing these derived values establishes the total baseline variance (100%), which is required to calculate individual component contribution percentages.

Line: `self.explained_variance_ratio_ = eigenvalues[:self.n_components] / total_variance`

1. Underlying Mechanics & Arguments:

- Truncates the converted eigenvalue vector to the target component count K and normalizes it against the total variance scalar.

2. Structural & Shape Transformation:

- **Input:** eigenvalues slice of shape `(K, )`.
- **Output:** `self.explained_variance_ratio_` array of shape `(K, )`.

3. BehaviorAL Alternatives:

- Modifying the slice parameter to a value greater than D will cause an empty slice or generate runtime alignment errors during assignment.

4. **Algorithmic & Math Justification:**

- This step calculates the relative variance profile for the SVD pipeline, ensuring the final metrics match the output of the eigenvalue pipeline.

Part 5: Error Handling & Data Projection

Lines:

```
else:  
    raise ValueError("method must be 'eig' or 'svd'")
```

1. **Underlying Mechanics & Arguments:**

- An error handling block designed to catch invalid configuration states. It stops execution by raising a `ValueError` with an informative message.

2. **Structural & Shape Transformation:**

- **Input:** Unmatched string state execution.
- **Output:** Breaks the runtime execution flow. No arrays are transformed because processing stops immediately.

3. **BehaviorAL Alternatives:**

- If this block used a silent fallback instead (e.g., `self.method = "svd"`), the code would continue running without throwing an error, but it might obscure user setup mistakes.

4. **Algorithmic & Math Justification:**

- Enforcing strict hyperparameter inputs prevents execution errors further down the pipeline, ensuring that all transformations rely on valid mathematical methods.

Line: `def transform(self, X):`

1. **Underlying Mechanics & Arguments:**

- Defines the projection method used to map new data into the learned low-dimensional coordinate space.
- `X`: The target data matrix to be transformed.

2. **Structural & Shape Transformation:**

- **Input:** Matrix of shape (M, D) , where M can be any number of samples, but D must match the feature count processed during the fitting phase.
- **Output:** Projected matrix space array.

3. BehaviorAL Alternatives:

- If a user passes a matrix with a different number of features (e.g., shape `(M, D+1)`), the matrix multiplication step will fail with a dimension mismatch error.

4. Algorithmic & Math Justification:

- The transformation phase applies the geometric basis rotation learned during training to new data points, projecting them onto the principal axes.

Line: `X_centered = X - self.mean`

1. Underlying Mechanics & Arguments:

- Subtracts the stored empirical mean vector (`self.mean`) from the input matrix `X`. This shifts the new data relative to the training center of mass.

2. Structural & Shape Transformation:

- **Inputs:** `X` of shape `(M, D)`; `self.mean` of shape `(D, )`.
- **Output:** `X_centered` of shape `(M, D)`.

3. BehaviorAL Alternatives:

- Using a local mean (`np.mean(X, axis=0)`) instead of the stored training mean (`self.mean`) would incorrectly center new test batches relative to themselves. This distorts their spatial position relative to the learned principal components.

4. Algorithmic & Math Justification:

- New test data must be centered using the original training center of mass to ensure that the learned coordinates remain valid and geometrically aligned:

$$X_{\text{test, centered}} = X_{\text{test}} - \mu_{\text{training}}$$

Line: `return np.dot(X_centered, self.components.T)`

1. Underlying Mechanics & Arguments:

- `np.dot(...)`: Performs standard matrix multiplication.
- `self.components.T`: Transposes the components matrix back to its original column orientation for the dot product operation.

2. Structural & Shape Transformation:

- **Inputs:** `X_centered` of shape `(M, D)`; `self.components.T` of shape `(D, K)`.

- **Output:** Returns a matrix of shape `(M, K)` and type `float64`.

3. BehaviorAL Alternatives:

- Forgetting to transpose (`self.components`) sets up an impossible matrix multiplication: `(M, D) x (K, D)`. This will fail immediately with a shape alignment error: `ValueError: shapes not aligned`.

4. Algorithmic & Math Justification:

- This step evaluates the linear projection equation:

$$Z = X_{\text{centered}} \cdot W^T$$

Multiplying the data matrix by the column-orthogonal basis matrix calculates the scalar coordinates of each sample along the new principal axes, completing the dimensionality reduction.

Part 6: Execution & Visualization Pipeline

Line: `if __name__ == "__main__":`

1. Underlying Mechanics & Arguments:

- A standard Python boilerplate check. It evaluates whether the script is being run directly as a main program rather than being imported as a module.

2. Structural & Shape Transformation:

- Returns a boolean primitive (`True` or `False`). Does not perform any array or matrix transformations.

3. BehaviorAL Alternatives:

- If omitted, the execution block will run even if the script is imported elsewhere as a utility class, which can trigger unintended evaluation loops.

4. Algorithmic & Math Justification:

- Separates class architecture declarations from executable simulation routines, ensuring clean code isolation.

Line: `data = load_wine()`

1. Underlying Mechanics & Arguments:

- Calls the scikit-learn dataset utility to load the classic Wine recognition dataset into memory as a dictionary-like container object.

2. Structural & Shape Transformation:

- **Input:** Void execution argument.

- **Output:** Returns a `Bunch` container object holding data tensors and metadata attributes.

3. BehaviorAL Alternatives:

- If the dataset package is missing or fails to load, execution stops here with an `ImportError` or `NameError`.

4. Algorithmic & Math Justification:

- Provides a real-world multi-class dataset with 13 features, making it an excellent benchmark for testing custom PCA implementations.

Lines:

```
X = data.data  
y = data.target
```

1. Underlying Mechanics & Arguments:

- Extracts the feature matrix data array (`.data`) and corresponding class target label vector (`.target`) from the dataset container.

2. Structural & Shape Transformation:

- **Outputs:**
 - `X`: Matrix of shape `(178, 13)` and type `float64`.
 - `y`: Integer array of shape `(178,)` containing class tracking indexes (0, 1, or 2).

3. BehaviorAL Alternatives:

- If `X` is sliced or modified incorrectly into a 1D layout, downstream data scaling operations will fail due to shape mismatch errors.

4. Algorithmic & Math Justification:

- Establishes the clear matrix definitions (X : Samples vs. Features; y : Evaluation Labels) required for our mathematical pipelines.

Line: `scaler = StandardScaler()`

1. Underlying Mechanics & Arguments:

- Instantiates a scikit-learn data preprocessor designed to normalize features by removing the mean and scaling to unit variance.

2. Structural & Shape Transformation:

- **Before:** Unbound utility reference.

- **After:** Creates a standardization instance template in memory.

3. BehaviorAL Alternatives:

- Skipping this scaling step means features with larger raw values will dominate the variance calculations, skewing the principal components toward those specific variables.

4. Algorithmic & Math Justification:

- Because PCA is highly sensitive to the relative scaling of features, standardization is necessary to give all variables equal weight. It normalizes each feature to a mean of 0 and a variance of 1:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Line: `X_scaled = scaler.fit_transform(X)`

1. Underlying Mechanics & Arguments:

- Computes the mean and standard deviation for each feature in X, then applies them to standardize the dataset.

2. Structural & Shape Transformation:

- **Input:** X of shape (178, 13).
- **Output:** X_scaled matrix of shape (178, 13) and type float64.

3. BehaviorAL Alternatives:

- Passing categorical variables or string types into this pipeline will break the mathematical calculations, causing a fatal conversion error.

4. Algorithmic & Math Justification:

- Transforms the data into a standard space where variance can be analyzed uniformly across all dimensions, removing scale bias.

Line: `pca = PCA(n_components=8, method="svd")`

1. Underlying Mechanics & Arguments:

- Instantiates the custom PCA class to reduce the data down to 8 dimensions using the SVD pipeline.

2. Structural & Shape Transformation:

- Creates an instance of the class with `self.n_components = 8` and `self.method = "svd"`.

3. BehaviorAL Alternatives:

- Setting `n_components=14` (which exceeds the original 13 features) will cause slicing issues or create empty columns in the downstream linear decomposition blocks.

4. Algorithmic & Math Justification:

- Setting $K = 8$ allows us to analyze how variance is distributed across multiple components, which is helpful for evaluating scree plots and cumulative variance thresholds.

Line: `pca.fit(X_scaled)`

1. Underlying Mechanics & Arguments:

- Runs the internal SVD-driven optimization loop on the standardized matrix `X_scaled`.

2. Structural & Shape Transformation:

- **Input:** `X_scaled` of shape `(178, 13)`.
- **Internal State Changes:**
 - `self.mean` is set to a shape of `(13, )`.
 - `self.components` is set to a shape of `(8, 13)`.
 - `self.explained_variance_ratio_` is set to a shape of `(8, )`.

3. BehaviorAL Alternatives:

- If an unscaled matrix is passed instead, the fitting process will still run, but the resulting components will be biased toward features with larger numerical ranges.

4. Algorithmic & Math Justification:

- This step extracts the primary directional vectors from the standardized dataset, mapping its structural variance pattern.

Line: `X_projected = pca.transform(X_scaled)`

1. Underlying Mechanics & Arguments:

- Projects the standardized matrix onto the learned 8-dimensional coordinate system.

2. Structural & Shape Transformation:

- **Input:** `X_scaled` of shape `(178, 13)`.
- **Output:** `X_projected` matrix of shape `(178, 8)` and type `float64`.

3. BehaviorAL Alternatives:

- If this method is called before running `.fit()`, it will crash with an `AttributeError` because `self.components` hasn't been initialized with a valid array yet.

4. **Algorithmic & Math Justification:**

- Reduces the dimensionality of the dataset from 13 to 8 while retaining as much variance as possible, compressing the feature space.

Line: `print("Explained variance ratio (PC1 & PC2):",
pca.explained_variance_ratio_)`

1. **Underlying Mechanics & Arguments:**

- Outputs the variance ratio array to the console to show how much information each principal component captures.

2. **Structural & Shape Transformation:**

- Reads from the `self.explained_variance_ratio_` array of shape `(8,)` and prints it as a text string.

3. **BehaviorAL Alternatives:**

- If the calculations are broken, this will print `nan` or display values outside the valid `0.0` to `1.0` range.

4. **Algorithmic & Math Justification:**

- Provides a quick diagnostic check to verify that the variance values are correctly sorted in descending order, confirming the algorithm is working as intended.

Line: `print("Total variance captured:",
sum(pca.explained_variance_ratio_))`

1. **Underlying Mechanics & Arguments:**

- Sums the individual variance ratios to calculate the total variance captured by the 8 principal components.

2. **Structural & Shape Transformation:**

- Sums the 8 elements of `self.explained_variance_ratio_` and prints the resulting scalar value.

3. **BehaviorAL Alternatives:**

- If the total sum exceeds `1.0`, it indicates a normalization error in the variance calculation pipeline.

4. **Algorithmic & Math Justification:**

- Quantifies exactly how much information is retained after reducing the dataset from 13 features down to 8 dimensions.

Part 7: Data Visualization Subsystem

Lines:

```
plt.figure()  
plt.scatter(X_projected[:, 0], X_projected[:, 1], c=y,  
cmap="viridis")
```

1. Underlying Mechanics & Arguments:

- `plt.figure()`: Creates a new figure window in memory for plotting.
- `X_projected[:, 0]`: Slices out the coordinates for the first principal component (PC1) to use as the x-axis values.
- `X_projected[:, 1]`: Slices out the coordinates for the second principal component (PC2) to use as the y-axis values.
- `c=y`: Maps the plot points to the class labels (`y`) to color-code the samples.
- `cmap="viridis"`: Applies a continuous color map gradient to visually distinguish the classes.

2. Structural & Shape Transformation:

- **Inputs**: Two distinct 1D slice arrays of shape `(178, )` derived from `X_projected`; color tracking map vector `y` of shape `(178, )`.
- **Output**: Generates a 2D scatter plot in the active figure window.

3. BehaviorAL Alternatives:

- If the inputs are sliced incorrectly as `X_projected[0, :]`, the plot will attempt to graph the features of the first sample instead of plotting the coordinates of all samples across the two principal components, causing a shape mismatch error.

4. Algorithmic & Math Justification:

- This step visualizes the dataset along its two primary axes of variance, allowing you to check if the classes are linearly separable in the new lower-dimensional space.

Lines:

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")  
plt.title("PCA on Wine Dataset")  
plt.show()
```

1. **Underlying Mechanics & Arguments:**

- Applies text labels and a title to the plot axes, then calls `plt.show()` to render the completed visualization to the display interface.

2. **Structural & Shape Transformation:**

- Modifies the metadata attributes of the active plot object and displays the figure.

3. **BehaviorAL Alternatives:**

- If running in a headless environment without a display server, calling `plt.show()` can cause a runtime warning or error if a non-interactive backend is active.

4. **Algorithmic & Math Justification:**

- Labels the axes clearly so you can easily interpret the distribution and alignment of the data points along the principal components.